Developer Guide

How the source code works - read this before changing it

Document	Developer Guide
Project	SailPoint IdentityIQ - HR CSV authoritative source, lifecycle and LDAP ApacheDS provisioning
Version	1.1
Date	23-10-2026
Author	Ashok Kumar
Source	github.com/ashok8033/Study - branch main, folder SailPointIIQ-HR-CSV

1. Who this guide is for

This guide explains the source code of the SailPoint IdentityIQ (IIQ) HR CSV and LDAP ApacheDS implementation so a developer can understand it, debug it and extend it safely. It assumes you know Java; it explains the IIQ and BeanShell concepts you need along the way. The HLD and LLD describe *what* the solution does; this guide explains *how the code does it*.

Read in this order

- Section 2-3: what is in the repository and the IIQ concepts used.
- Section 4: when IIQ runs each piece of code (the most important section).
- Sections 5-10: walk-through of every file, with the key code.
- Sections 11-13: how to extend it, coding conventions, debugging.

2. Repository layout

```
SailPointIIQ-HR-CSV/
  README.md                       setup notes, kept in step with the code
  data/HR_Employees.csv           sample HR feed (copy to C:/SailPoint/hrfeed/)
  ldif/                           ApacheDS entries: OUs and groups
  docs/                           HLD, LLD, this guide and their generator scripts
  config/
    init-HR-CSV.xml               master import file - imports everything in order
    Application/                  HR Authoritative CSV, LDAP ApacheDS
    Rule/                         all BeanShell code (22 rules)
    Bundle/                       roles (business + IT)
    ManagedAttribute/             the Office365Access entitlement
    Workflow/  IdentityTrigger/  EmailTemplate/   lifecycle event, workflows, email
    QuickLink/                    Emergency Identity quick link, capability, scope
    TaskDefinition/               aggregation and refresh tasks
```

Naming convention. Every file is `<ObjectType>-<Area>-<Name>.xml` and the object inside is named `<Area> - <Name>`, for example `Rule-HR_CSV-Correlation.xml` holds the rule **HR CSV - Correlation**. Areas: **HR CSV** (the HR source), **HR Lifecycle** (status and events), **LDAP ApacheDS** (the target), **Emergency Identity** (the emergency quick link).

Every file is a SailPoint XML object. Each starts with `<!DOCTYPE <Root> PUBLIC "sailpoint.dtd" "sailpoint.dtd">`; files holding several objects use the root `<sailpoint>`. IIQ validates the XML against its DTD on import, so element names and nesting must match exactly.

3. IIQ concepts used in the code

Concept	What it is	Where you see it
Application	Connection to a system + its account schema	HR Authoritative CSV, LDAP ApacheDS
ResourceObject	One account as read from the source, before IIQ stores it	Customization rules (object)
Link	An account stored in IIQ, attached to an identity	Set Account Status, Display Name rule
Identity	A person (the 'identity cube')	Every identity rule, workflow
ObjectConfig 'Identity'	Definition of identity attributes and their sources / targets	Setup Identity Mappings
Rule	BeanShell script IIQ calls at a defined point (by type)	config/Rule
ProvisioningPlan	List of changes to make on target accounts (AccountRequest / AttributeRequest)	Before / After Provisioning
Bundle	A role: business (assigned / requested) or IT (holds entitlements)	config/Bundle
IdentityTrigger	Lifecycle event: fires when something about an identity changes	HR Lifecycle - Status Change
Workflow	Steps run on an event; steps can call rules	HR Lifecycle - Status Change Notification
SailPointContext	The API handle every rule receives as 'context'	context.getObjectByName(...), context.saveObject(...)

BeanShell. Rules are written in BeanShell: Java syntax, interpreted at run time. Things that differ from Java:

- Variables can be untyped; a missing variable can be tested with `void == name`.
- Inputs (context, log, identity, object ...) are injected as variables - the <Signature> only documents them.
- A **library rule** holds shared functions; other rules include it with <ReferencedRules> and call its functions directly.
- Errors surface at run time only, so test every change by running the rule (see section 12).

4. When does each piece of code run?

Most confusion comes from not knowing when IIQ calls a rule. Three processes drive everything.

4.1 HR aggregation (task: HR Authoritative CSV - Account Aggregation)

```
for each row in HR_Employees.csv:
  1. DelimitedFileConnector parses the row      -> ResourceObject
  2. HR CSV - Customization                      trim, validate dates, status, IIQDisabled
  3. HR CSV - Correlation                        which identity owns it? (identityName = employeeNumber)
  4. HR CSV - Identity Creation                  ONLY if no identity found: create it
  5. Link saved; identity attributes promoted    (mappings from ObjectConfig Identity)
  6. HR CSV - Manager Correlation                managerId -> manager identity
```

4.2 Identity refresh (task: HR Lifecycle - Daily Identity Refresh)

```
for each identity:
  1. promoteAttributes      sources + global rules:
                              HR Lifecycle - Status Attribute  -> status
                              HR CSV - Display Name             -> displayName
  2. processTriggers        status changed? -> IdentityTrigger -> Workflow (section 8)
  3. correlateEntitlements  detect roles already met by LDAP groups
  4. provision              run birthright assignment rules; add / remove roles;
                            build a ProvisioningPlan for missing access (section 4.3)
  5. synchronizeAttributes  push displayName to the LDAP account (attribute target)
```

4.3 Provisioning to LDAP (inside the refresh, or after an approved request)

```
ProvisioningPlan for LDAP ApacheDS
  1. Create Account Policy fills a new account (uid, dn, cn, mail, password ...)
  2. LDAP ApacheDS - Before Provisioning   take 'groups' out, save as hrGroupsAdd / hrGroupsRemove
  3. LDAPConnector                         ADD or MODIFY the account entry
  4. LDAP ApacheDS - After Provisioning    add / remove uniqueMember on each group (JNDI)
```

4.4 LDAP aggregation (task: LDAP ApacheDS - Account Aggregation)

```
for each inetOrgPerson under ou=people:
  1. LDAPConnector reads it                -> ResourceObject
  2. LDAP ApacheDS - Customization         add groups + account status (JNDI)
  3. LDAP ApacheDS - Correlation           identityName = employeeNumber
```

5. HR source code

5.1 HR CSV - Customization (ResourceObjectCustomization)

Input `object` is the parsed CSV row; whatever it returns is what IIQ stores. It never throws: bad data is logged and defaulted, so one bad row cannot stop the aggregation.

```
// 1. trim every String attribute; "" becomes null
// 2. parse startDate / endDate with SimpleDateFormat("dd-MM-yyyy"), setLenient(false)
// 3. isManager -> "Y" / "N" (blank or invalid -> "N", logged)
// 4. status, compared by WHOLE DAY (midnight today):
if (end != null && end.before(today))         status = "Terminated";
else if (start != null && start.after(today)) status = "PreHire";
else                                          status = "Active";
object.setAttribute("status", status);
object.setAttribute("inactive", !"Active".equals(status));
object.setAttribute("IIQDisabled", Boolean.valueOf("Terminated".equals(status)));
```

IIQDisabled is a reserved attribute name: IIQ uses it to mark the account (Link) disabled, which drives the red/green icon on Application Accounts.

5.2 HR CSV - Correlation and Manager Correlation

Both return a Map telling IIQ which identity to use. Returning `identityName` matches on the identity's name, which is always searchable.

```
Map result = new HashMap();
String empNo = account.getStringAttribute("employeeNumber");
if (empNo != null && empNo.trim().length() > 0) result.put("identityName", empNo.trim());
return result;      // empty map = no match -> Identity Creation rule runs
```

Why not identityAttributeName = employeeNumber? That needs employeeNumber to be a *searchable* identity attribute (database column). It is not, and the first version failed with `could not resolve property: employeeNumber`. Naming identities after the employee number avoids the schema change.

5.3 HR CSV - Identity Creation (IdentityCreation)

Runs once, only for a new identity. It sets the name - this is what makes correlation work - and first values.

```
identity.setName(empNo);                    // identity name = employeeNumber
identity.setFirstname(first); identity.setLastname(last);
identity.setDisplayName(first + " " + last);  // later kept current by HR CSV - Display Name
identity.setType("Employee".equalsIgnoreCase(type) ? "employee" : "contractor");
```

6. Identity attributes and calculated values

6.1 Global rules (IdentityAttribute)

A global rule calculates an identity attribute on every refresh. IIQ passes `identity` and `oldValue`; the return value becomes the new value. Both global rules read the **HR link** rather than other identity attributes, because the order in which attributes are promoted is not guaranteed.

```
Application app = context.getObjectByName(Application.class, "HR Authoritative CSV");
Link hr = (app == null) ? null : identity.getLink(app);
Object startValue = (hr != null) ? hr.getAttribute("startDate") : identity.getAttribute("startDate");
...
if (start == null && end == null) return oldValue;   // no data: keep what we had
```

Rule	Returns
HR Lifecycle - Status Attribute	Terminated if endDate passed; else Disabled while emergencyDisable is true; else PreHire / Active (same day logic as section 5.1)
HR CSV - Display Name	firstName + ' ' + lastName from the HR link
Emergency Identity - Inactive Attribute	true while emergencyDisable is set, otherwise the HR link's inactive

6.2 HR CSV - Setup Identity Mappings (one-time setup rule)

Creates the identity attributes in ObjectConfig **Identity** through the API instead of importing XML, because importing an ObjectConfig replaces the whole object. It is **idempotent**: it checks before it adds, so running it again is safe.

```
String[][] MAPPINGS = {   // { identity attribute, display name, type, HR attribute }
    { "employeeNumber", "Employee Number", "string", "employeeNumber" },
    { "manager", null, "sailpoint.object.Identity", "managerId" }, ... };
ObjectAttribute oa = oc.getObjectAttribute(idAttr);          // create if missing
AttributeSource src = new AttributeSource(); src.setApplication(app); src.setName(appAttr);
statusAttr.setRule(statusRule);                             // global rule
AttributeTarget t = new AttributeTarget(); t.setApplication(ldapApp); t.setName("displayName");
context.saveObject(oc); context.commitTransaction();
```

An attribute that already has a global rule (for example **inactive** after **Emergency Identity - Setup**) is skipped: adding a source to it would bypass the rule.

To add a new HR attribute: add a column to the CSV, an AttributeDefinition to the HR application schema, one line to MAPPINGS, then re-run the setup rule and a refresh.

7. LDAP ApacheDS code

7.1 Why the LDAP rules exist

IIQ treats `groups` as an *entitlement* on the account. The LDAP connector sent it to ApacheDS as an LDAP attribute, which ApacheDS rejected (`NO_SUCH_ATTRIBUTE ... groups`). Group membership is therefore done by rules that talk to ApacheDS directly with **JNDI** (javax.naming). The same connection code also reads and sets account status, which the connector does not support for ApacheDS.

7.2 LDAP ApacheDS - Library (shared functions)

Function	What it does
ldapConnect(context, app)	Opens a JNDI connection with the application's host, port, user and (decrypted) password
ldapSetMember(ldap, groupDn, userDn, add)	ADD / REMOVE uniqueMember; 'already member' / 'not member' count as success
ldapSetDisabled(ldap, userDn, disable)	REPLACE pwdAccountLockedTime = 000001010000Z, or REMOVE it
ldapGroupsOf(ldap, userDn)	Search ou=groups for (uniqueMember=userDn); returns group DNs
ldapReadAttributes(ldap, dn, attrIds)	Read named attributes - needed for operational ones like pwdAccountLockedTime
ldapFormatTime(value)	20261013101500Z -> 13-10-2026 10:15:00 UTC

Rules that use it declare:

```
<ReferencedRules>
  <Reference class="sailpoint.object.Rule" name="LDAP ApacheDS - Library"/>
</ReferencedRules>
```

7.3 Before / After Provisioning - how group changes travel

The two rules hand data to each other through the **arguments** of the AccountRequest, which survive from before to after the connector call.

```
// BEFORE: remove "groups" so ApacheDS accepts the request; remember the change
if ("groups".equals(req.getName())) {
    (Remove ? removes : adds).addAll(values);
    it.remove();
}
args.put("hrGroupsAdd", adds);  args.put("hrGroupsRemove", removes);

// AFTER: only if the account request did not fail
if (r != null && r.isFailed()) continue;
String userDn = acct.getNativeIdentity();     // or the "dn" attribute request on a Create
for (g : adds)    ldapSetMember(ldap, g, userDn, true);
for (g : removes) ldapSetMember(ldap, g, userDn, false);
```

Both are attached as **application attributes**, not elements (the Application DTD has no <BeforeProvisioningRule> element):

```
<entry key="beforeProvisioningRule" value="LDAP ApacheDS - Before Provisioning"/>
<entry key="afterProvisioningRule"  value="LDAP ApacheDS - After Provisioning"/>
```

7.4 LDAP ApacheDS - Customization (read-back)

Runs for every account during LDAP aggregation. It keeps one JNDI connection in the aggregation `state` map so it does not reconnect for every account.

```
DirContext ldap = (DirContext) state.get("hrLdapContext");
if (ldap == null) { ldap = ldapConnect(context, application); state.put("hrLdapContext", ldap); }
object.setAttribute("groups", ldapGroupsOf(ldap, dn));
String locked = ldapReadAttributes(ldap, dn, STATUS_ATTRS).get("pwdAccountLockedTime");
status = locked == null ? "Active" : ("000001010000Z".equals(locked) ? "Disabled" : "Locked");
object.setAttribute("IIQDisabled", "Disabled".equals(status));
object.setAttribute("IIQLocked",   "Locked".equals(status));
```

7.5 LDAP ApacheDS - Set Account Status

Called by the workflow (and by the catch-up rule) with `identityName` and `action` = Disable / Enable. It changes ApacheDS, then updates the Link at once so the UI is right without waiting for the next aggregation.

```
ldapSetDisabled(ldap, link.getNativeIdentity(), disable);
link.setAttribute("accountStatus", disable ? "Disabled" : "Active");
link.setAttribute("pwdAccountLockedTime", disable ? "000001010000Z" : null);
link.setAttribute("IIQDisabled", Boolean.valueOf(disable));
context.saveObject(link); context.commitTransaction();
```

7.6 Create Account Policy

Lives inside the LDAP application (<ProvisioningForms>, Form type="Create"). Each Field has a Script; `identity` is available. A field can use another field's value as a variable when it declares `dependencies="uid"` - that is how dn and mail are built from uid. uid is made unique by counting existing LDAP links with the same DN.

8. Lifecycle event and workflow

The IdentityTrigger fires during a refresh with *processTriggers* when **status** changes, and starts the workflow with `identityName`, `event` (IdentityChangeEvent: old and new identity) and `trigger`.

```
Start -> Collect Details
         | newStatus == Terminated and oldStatus != Terminated  -> Disable LDAP
         | oldStatus == Terminated and newStatus == Active
         |   and emergencyDisable is not true                   -> Enable LDAP
         | otherwise                                            -> Check Notify
Disable LDAP / Enable LDAP: action="rule:LDAP ApacheDS - Set Account Status"
                            resultVariable="ldapResult"  -> Check Notify
Check Notify: oldStatus and newStatus set and different -> Send Email, else Stop
Send Email:   action="call:sendEmail", template "HR Lifecycle - Status Change"
```

- **explicitTransitions="true"**: the first Transition whose script returns true is taken; a Transition without a script is the default and must come last.
- Workflow variables are plain BeanShell variables in transition scripts; a step script sets them with `workflow.put("name", value)`.
- Each <Arg> of the sendEmail step becomes a Velocity variable in the template ($!oldStatus ...). `$!` prints nothing instead of the literal name when the value is null.
- No email the first time status is set (oldStatus null) - otherwise the first refresh would email everyone.

9. Emergency Identity quick link and workflow

9.1 Access: quick link, DynamicScope, capability

The QuickLink (action `workflow`) is bound to the DynamicScope **Emergency Identity Administrators**, whose IdentitySelector matches `capabilities.name` SystemAdministrator or EmergencyIdentityAdministrator. The scope only hides the link, so the workflow's **Validate Request** step checks the launcher's effective capabilities again:

```
List caps = me.getCapabilityManager().getEffectiveCapabilities();
for (Object c : caps) {
    String n = ((Capability) c).getName();
    if ("SystemAdministrator".equals(n) || "EmergencyIdentityAdministrator".equals(n)) admin = true;
}
```

9.2 The form

The form is the <Form> inside the first Approval step (owner = launcher, workItemType Form, workflow variable `transient` = true so nothing is stored while it is open). The identity field has `postBack="true"`; the other fields are `dynamic="true"` and recalculate from it on every post back:

```
Object v = form.getField("targetIdentity").getValue();          // id or name
Identity id = (v instanceof Identity) ? (Identity) v : context.getObject(Identity.class, v.toString());
boolean on = "true".equalsIgnoreCase(String.valueOf(id.getAttribute("emergencyDisable")));
return on ? "Enable" : "Disable";   // AllowedValuesDefinition offers only this value
```

The Approval's `return` attribute copies targetIdentity, requestedAction and justification back into workflow variables; its AfterScript sets `cancelled` when the work item was not completed with Submit.

9.3 Update, provisioning, audit

- **Update Identity** takes an identity lock (ObjectUtil.lockIdentity) so it cannot clash with a running refresh, sets emergencyDisable, runs the status and inactive global rules for the new value and unlocks (which saves and commits).
- **Emergency Identity - Provision Accounts** collects the work first and then acts, because the LDAP rule commits. LDAP ApacheDS goes through **LDAP ApacheDS - Set Account Status**; other applications with the ENABLE feature get an AccountRequest (Disable / Enable) executed by `new Provisioner(context).execute(plan)`, with no approvals. It returns a Map: status (SUCCESS / PARTIAL / FAILED / NOTHING_TO_DO) and details (one line per account).
- **Emergency Identity - Audit** runs on every path, including DENIED and FAILED, and saves an AuditEvent directly (not through Auditor.log), so it is written even if the action is not enabled in AuditConfig.
- **Show Result** is a second, read-only form that shows workflowResult and the detail lines.

10. Roles, entitlements and tasks

Roles are data, not code. A business role either has a **Selector** (assignment rule) or none (requestable). It points to an IT role through <Requirements>; the IT role's Profile holds the LDAP group.

```
<Selector><IdentitySelector><MatchExpression and="true">
```

```
  <MatchTerm name="status" value="Active"/>
  <MatchTerm name="employeeType" value="Contractor"/>
</MatchExpression></IdentitySelector></Selector>

<Profile> ... <Filter operation="CONTAINS_ALL" property="groups">
  <Value><List><String>cn=Contractors,ou=groups,dc=example,dc=com</String></List></Value>
</Filter></Profile>
```

Task option (Daily Identity Refresh)	Effect
promoteAttributes	Run sources and global rules (status, displayName)
processTriggers	Fire lifecycle events (the workflow)
correlateEntitlements	Detect IT roles already satisfied by LDAP groups
provision	Evaluate assignment rules and provision / deprovision roles
synchronizeAttributes	Push attributes that have targets (displayName -> LDAP)

11. How to extend

Change	Files to touch
New HR column	data CSV header; HR application schema; MAPPINGS in Setup Identity Mappings; ShowOnIdentityPage ADD list; re-run both setup rules + refresh
New calculated identity attribute	New IdentityAttribute rule; attach with setRule() in Setup Identity Mappings
New birthright LDAP group	LDIF entry; IT role (Profile with the group DN); business role with a Selector; refresh
New requestable access	LDIF group; ManagedAttribute (requestable="true"); IT role; business role without Selector
Sync another attribute to LDAP	LDAP schema attribute; AttributeTarget on the identity attribute (like displayName)
New lifecycle action	Add a step + Transition in the workflow; put the logic in a rule, call it with action="rule:..."
Different OU / directory layout	dn field + uniqueness DN in the Create policy; searchDN entries; group DNs in IT roles, ManagedAttribute, Library (ldapGroupsOf); LDIF
Another target system	New Application XML; correlation rule; IT roles with its entitlements
Emergency disable for another target	Give its connector the ENABLE feature (picked up automatically), or add a branch like the LDAP one in Emergency Identity - Provision Accounts

12. Coding conventions and gotchas

- **Never import ObjectConfig, UIConfig or Configuration objects wholesale** - they replace everything. Change them with an idempotent setup rule (see Setup Identity Mappings / ShowOnIdentityPage).
- **Every file needs a DOCTYPE** whose root matches the root element, or the import fails.
- **Follow the DTD.** Examples that bit us: HandlerParameters must wrap its Map in <Attributes>; provisioning rules are application attributes, not elements.
- **Correlate on identity name** unless an attribute is made searchable.
- **Rules must not throw during aggregation** - catch, log with a clear prefix ('HR CSV' / 'LDAP ApacheDS:') and continue.
- **Make utility rules idempotent** and print a report of what they did; they are run by hand.
- **Dates:** DD-MM-YYYY strings everywhere, parsed with setLenient(false), compared by whole day.
- **Close JNDI connections** in finally, except the aggregation-scoped one kept in state.
- **Commit after saving** outside the normal pipeline: context.saveObject(x); context.commitTransaction();
- **Do not trust the UI for authorisation** - re-check rights in the workflow, as the emergency workflow does.

- **Audit every outcome**, including refused and failed requests.
- **Keep README, init-HR-CSV.xml and these docs in step** with any new file.

13. Debugging toolkit

Tool	Use it to
rule "HR CSV - Diagnose"	See mappings (database vs memory), the HR link, the identity values and the status rule result for one identity
rule "Aggregate Single Account"	Re-read one account (APP_NAME / ACCOUNT_ID) and print what the source returned
rule "HR Lifecycle - Send Test Email"	Check mail settings and send one email
Debug page (/identityiq/debug)	View raw XML of any object: Identity, Link, Rule, Workflow, IdentityTrigger
IIQ console: get Identity 1000	Same as above from the command line
log4j2.properties	Raise a logger to debug, e.g. sailpoint.connector.DelimitedFileConnector; reload logging from Debug
Advanced Analytics > Audit Search	Emergency Identity events: action EmergencyIdentity*, source = requestor, target = identity
Task Results	Counts (created / updated) and errors of aggregations and refreshes
Apache Directory Studio	Check entries; enable 'Fetch operational attributes' to see pwdAccountLockedTime

Edit - import - test loop: change the XML, check it is well formed, import that one file in the console (`import config/Rule/<file>.xml`), run the smallest thing that exercises it (a utility rule or Aggregate Single Account), read the output and the log.